

CompositeCurves User Manual

This manual is for designers, technical artists, gameplay programmers, and anyone authoring curve assets inside Unity.

What CompositeCurves Is

CompositeCurves lets you build a curve from multiple segments instead of forcing everything into one function.

Typical uses include:

- XP or level progression
- Damage falloff
- Economy scaling
- Spawn weighting
- Difficulty ramps
- Animation or timing helpers
- Balancing formulas with designer-controlled parameters

Creating a Curve Asset

1. In the Unity Project window, choose **Assets/Create/Composite Curves/Curve Definition**.
2. Name the asset.
3. Select it to open the custom inspector.

The asset stores:

- a list of segments
- how out-of-range values are handled
- any preset or custom variable values

Understanding a Segment

A segment is one piece of the full piecewise function.

Each segment has:

- Name
- Enabled
- Start X
- End X
- Start Bound
- End Bound
- Mode
- variables

If **Mode** is **Preset**, the segment uses one of the built-in curve families.

If **Mode** is **Custom**, the segment uses your custom expression and generated code.

Adding Segments

Use:

- **Add Preset Segment**
- **Add Custom Segment**

When a new segment is created:

- its **Start X** defaults to the **End X** of the last existing segment
- its **End X** defaults to **Start X + 1**

This only happens when the segment is created. It does not keep changing after that.

Built-In Presets

Available presets:

- Constant
- Linear
- Quadratic
- Cubic
- Sine
- Cosine
- Tangent
- QuadraticBezier
- CubicBezier

Common variable meanings

- Constant: **y**
- Linear: **m**, **c**
- Quadratic: **a**, **b**, **c**
- Cubic: **a**, **b**, **c**, **d**
- Trig: **amplitude**, **frequency**, **phase**, **offset**
- Bezier: **y0**, **y1**, **y2**, **y3** depending on order

Use **Apply Preset Defaults** if you want to restore the default variable list for a preset segment.

Custom Curves

Custom segments let you enter a single expression using:

- **x**
- your own named variables
- supported math aliases

Example expressions:

```
m * x + c  
pow(x, 2) + offset  
amplitude * sin(frequency * x + phase) + offset
```

Supported convenience aliases

Common aliases are translated to `Mathf` calls in generated code, including:

- `sin`
- `cos`
- `tan`
- `asin`
- `acos`
- `atan`
- `atan2`
- `sqrt`
- `pow`
- `abs`
- `min`
- `max`
- `clamp`
- `clamp01`
- `exp`
- `log`
- `log10`
- `floor`
- `ceil`
- `round`
- `sign`
- `pi`
- `deg2rad`
- `rad2deg`

Important rule

Custom expressions are not interpreted at runtime.

You must regenerate code when you change custom expressions or custom variable naming in a way that affects generated output.

Use:

- **Regenerate Custom Curves** in the inspector
- or **Tools/Composite Curves/Regenerate Generated Curves**

Variables

Each segment has a variable list.

You can:

- rename variables
- add variables
- remove variables
- change numeric values

Scroll editing

Variable value fields support mouse-wheel editing.

- Scroll over a numeric value to change it
- **Shift** uses larger steps
- **Ctrl** or **Cmd** uses finer steps

Bounds

Each segment has:

- **Start Bound**
- **End Bound**

Each can be:

- **Inclusive**
- **Exclusive**

This determines whether the exact endpoint belongs to that segment.

Examples:

- $[0, 10]$ means both ends are inclusive
- $[0, 10)$ means start inclusive, end exclusive
- $(0, 10]$ means start exclusive, end inclusive
- $(0, 10)$ means both exclusive

If two touching segments both include the same shared boundary, the earlier segment wins for that exact **x**.

Sorting Segments

Use **Sort Segments** to reorder the list by domain.

Sorting uses the segment range itself, not just the raw field order, so it handles reversed ranges more predictably.

Sorting is useful after:

- adding many segments out of order

- manually editing domains
- duplicating and then retargeting a segment

Preview Window

The inspector includes a curve preview.

It does not automatically refit every time the curve changes. This is intentional so the view stays stable while you edit bounds and values.

Preview controls

- Drag to pan
- Scroll to zoom
- **Shift+Scroll** to zoom X only
- **Alt+Scroll** to zoom Y only
- **Reset View** to refit manually

The preview shows:

- the current view range
- the detected data range

Outside Range Behavior

The **Outside Range** option controls what happens when **x** is outside every segment.

Modes:

- **ReturnZero**
- **ClampToEdge**
- **ExtrapolateNearestSegment**

Practical meaning

- **ReturnZero**: returns 0
- **ClampToEdge**: returns the nearest edge value
- **ExtrapolateNearestSegment**: evaluates using the nearest segment outside its normal range

Duplicating and Removing Segments

Each segment can be:

- duplicated
- removed

Duplicate is useful when:

- you want a similar preset with small parameter changes

- you want to split a domain into two nearby segments

Using Curves in Game Code

Reference a `CompositeCurveDefinition` from a `MonoBehaviour` or another `ScriptableObject`.

Example:

```
using CompositeCurves;
using UnityEngine;

public sealed class RewardCurveExample : MonoBehaviour
{
    [SerializeField] private CompositeCurveDefinition rewardCurve;

    public float EvaluateReward(float level)
    {
        return rewardCurve != null ? rewardCurve.GetValue(level) : 0f;
    }
}
```

Recommended Workflow

1. Create the curve asset.
2. Add broad preset segments first.
3. Adjust domains and bounds.
4. Tune variables.
5. Use custom segments only where presets are not enough.
6. Regenerate custom curves when needed.
7. Verify values in play mode or with your gameplay systems.

Troubleshooting

My custom curve returns zero

Possible reasons:

- the segment is disabled
- the queried `x` is outside the segment domain
- bounds exclude the exact `x` you are testing
- generated code has not been regenerated yet
- the expression is invalid and generated to a safe fallback

The preview does not refit after I changed bounds

That is expected.

Use `Reset View` if you want the preview to refit.

Sorting did not change evaluation the way I expected

Check:

- overlapping domains
- inclusive shared endpoints
- reversed **Start X** and **End X**

Sorting only changes order. It does not rewrite the actual ranges.

Related Documents

- Overview and quick reference: README.md
- Contributor and internal design notes: ARCHITECTURE.md